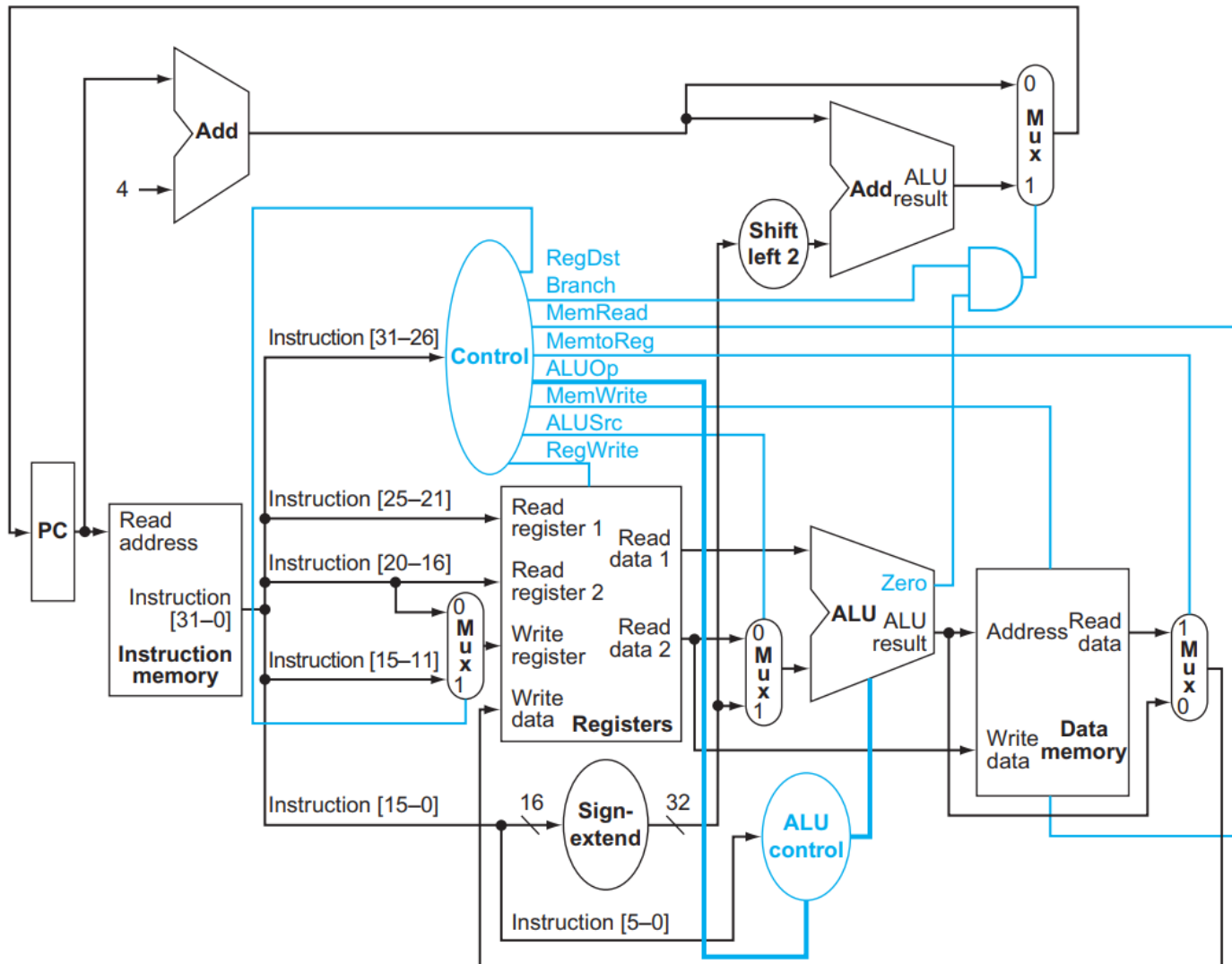


AULA 2 – VISÃO GERAL DA ARQUITETURA E DO CONJUNTO DE INSTRUÇÕES DO MIPS

Luiz Paulo de Assis Barbosa

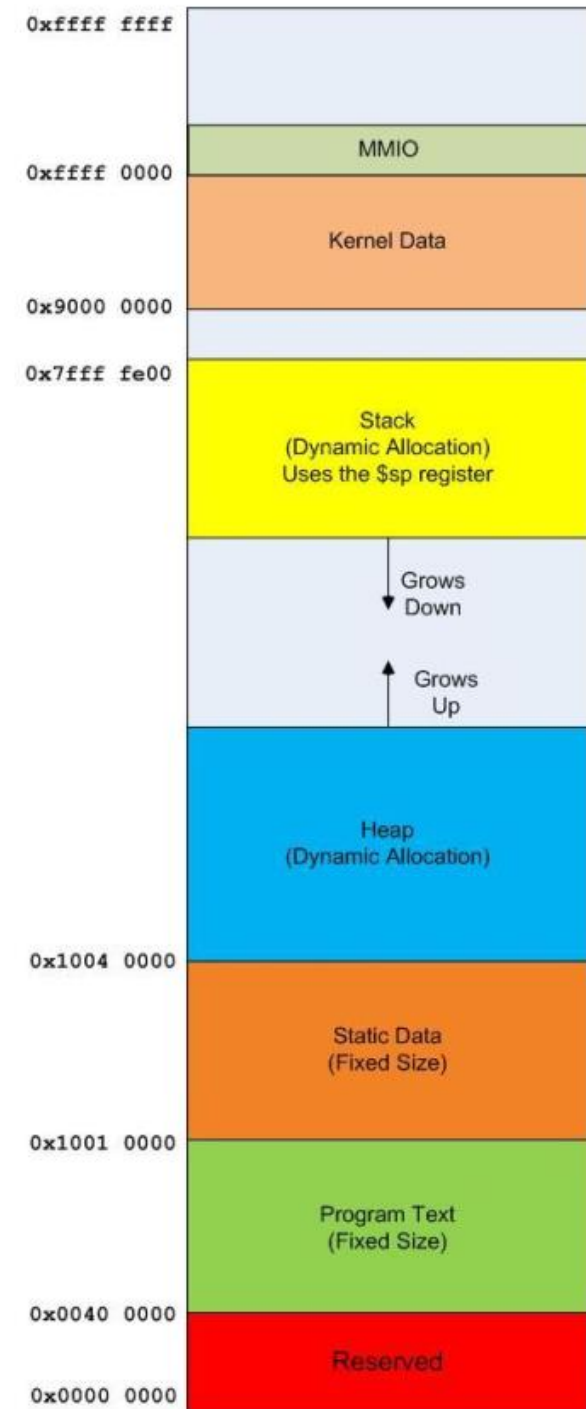
Visão Geral da Arquitetura MIPS

2



Memória do MIPS

- Reservado
 - ▣ usado pela plataforma MIPS
 - ▣ inacessível a programas
- Texto de Programa
 - ▣ onde o código de máquina é armazenado
 - ▣ 32 bits para cada instrução



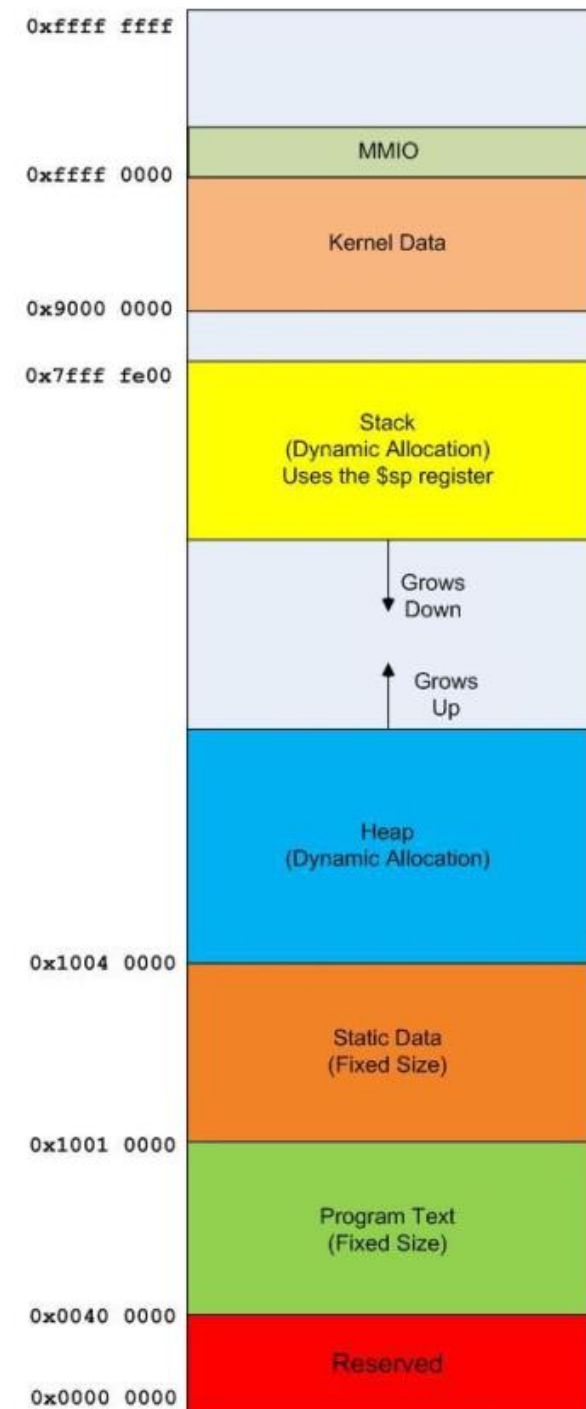
Memória do MIPS

□ Dados Estáticos

- ▣ onde ficam os dados oriundos do seguimento `.data` do programa
- ▣ o tamanho dos dados é fixo, não podem mudar durante a execução do programa

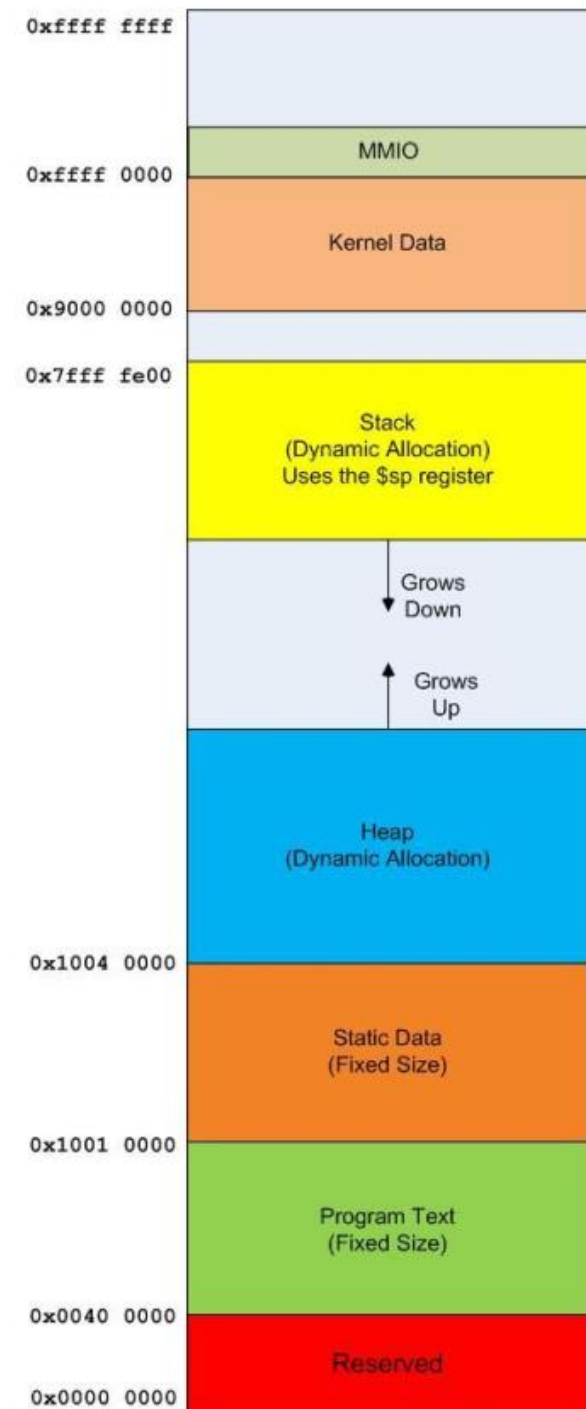
□ Heap

- ▣ usada para alocação dinâmica de memória em tempo de execução



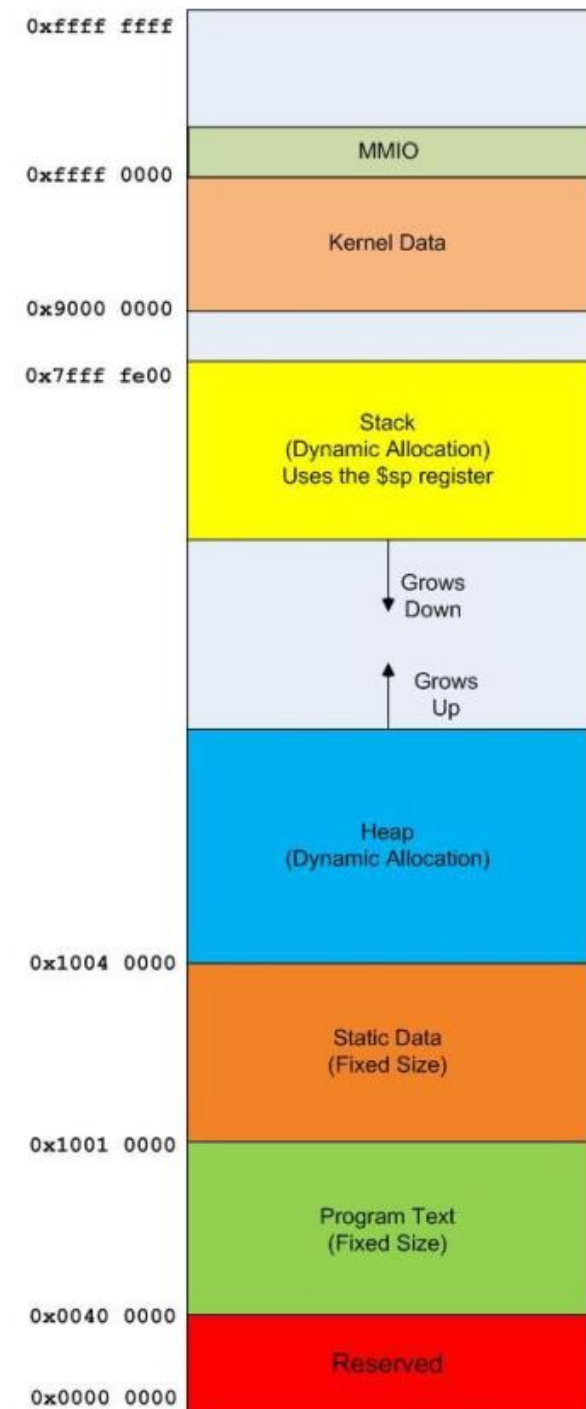
Memória do MIPS

- Pilha (stack)
 - ▣ usada para alocação dinâmica de dados para subprogramas
- Dados do Kernel
 - ▣ usada pelo sistema operacional
 - ▣ inacessível ao programa de usuário



Memória do MIPS

- MMIO (Memory Mapped I/O)
 - ▣ E/S mapeada em memória
 - ▣ usada para acessar dados que não estão na memória
 - ▣ mapeia endereço de dispositivos de E/S como drives de disco, monitores, etc



Registradores da CPU

Nome	Número	Uso
\$zero	\$0	Sempre contém zero
\$at	\$1	Assembler temporary
\$v0 - \$v1	\$2 - \$3	Retorno de valores de subprogramas
\$a0 – \$a3	\$4 - \$7	Passagem de argumento para subprogramas
\$t0 – \$t7	\$8 - \$15	Temporários
\$s0 – \$s7	\$16 - \$23	Salvos, valores devem ser preservados em chamadas de procedimentos
\$t8 – \$t9	\$24 - \$25	Temporários
\$k0 - \$k1	\$26 - \$27	Usados pelo SO
\$gp	\$28	Ponteiro global , usado para alocação na Heap
\$sp	\$29	Ponteiro de pilha
\$fp	\$30	Ponteiro de quadro
\$ra	\$31	Endereço de retorno

Tipos de dados

8

Tipo	Tamanho (em bytes)	Tamanho em bits
byte	1	Inteiro de 8 bits
halfword	2	Inteiro de 16 bits
word	4	Inteiro de 32 bits
float	4	Ponto flutuante de 32 bits
double	8	Ponto flutuante de 64 bits
ascii	-	String ASCII
asciiz	-	String ASCII terminada com \0 (null)
space <n>	n	<n> bytes de memória não inicializada (pode conter qualquer conteúdo)

Declarando Variáveis

#Example 1 - Declaração de variáveis e tipos de dados

```
.data
    _byte: .byte 'a'                # 1 byte
    _halfword: .half 53             # 2 bytes
    _word: .word 3                   # 4 bytes
    _float: .float 3.5               # 4 bytes
    _str1: .ascii "Hello World\n"   # 13 bytes
    _num1: .word 42                  # 4 bytes
    _arr1: .word 1, 2, 3, 4, 5       # Array de intereiros
    _arr2: .byte 'a', 'b', 'c', 'd'  # Array de chars
    _buffer: .space 20               # 20 bytes
    _num2: .word 16                  # 4 bytes

    .align 2                        #alinhamento de MEM

.text
.globl _main
    _main:
        li $v0, 4 #service 4, imprime a string guardada a partir do
                #endereço especificado em $a0
        la $a0, _str1 #load address, carrega em $a0 o endereço de _str1
        syscall
        li $v0, 10 #sevice 10, pára ou termina o programa
        syscall
```

Declarando Variáveis

10

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00350061	0x00000003	0x40600000	0x6c6c6548	0x6f57206f	0x0a646c72	0x00000000	0x0000002a
0x10010020	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x64636261	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000010	0x00000000	0x00000000	0x00000000	0x00000000

```
_byte: .byte 'a'           # 1 byte
```

Endereço: 0x10010000 (+0) ou 0x10010000

Conteúdo: 0x00350061

Interpretação: 0x61 (ASCII) -> a (caractere)

Declarando Variáveis

11

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00350061	0x00000003	0x40600000	0x6c6c6548	0x6f57206f	0x0a646c72	0x00000000	0x0000002a
0x10010020	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x64636261	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000010	0x00000000	0x00000000	0x00000000	0x00000000

```
_halfword: .half 53      # 2 bytes
```

Endereço: 0x10010000 (+2) ou 0x10010002

Conteúdo: 0x00350061

Interpretação: 0x0035, 0x35 -> 53 (decimal)

Declarando Variáveis

12

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00350061	0x00000003	0x40600000	0x6c6c6548	0x6f57206f	0x0a646c72	0x00000000	0x0000002a
0x10010020	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x64636261	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000010	0x00000000	0x00000000	0x00000000	0x00000000

```
_word: .word 3           # 4 bytes
```

Endereço: 0x10010000 (+4) ou 0x10010004

Conteúdo: 0x00000003

Interpretação: 0x00000003, 0x03 -> 3 (decimal)

Declarando Variáveis

13

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00350061	0x00000003	0x40600000	0x6c6c6548	0x6f57206f	0x0a646c72	0x00000000	0x0000002a
0x10010020	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x64636261	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000010	0x00000000	0x00000000	0x00000000	0x00000000

```
_float: .float 3.5          # 4 bytes
```

Endereço: 0x10010000(+8) ou 0x10010008

Conteúdo: 0x40600000

Interpretação:

0100 0000 0110 0000 0000 0000 0000 0000

senal = 0 (+)

expoente = 1000 0000 -> 128 - 127(excesso) = 1

mantissa = 110 0000 0000 0000 0000

conversão = 1,11 x 2¹

= 11,1 (binário) -> 3,5 (decimal)

Declarando Variáveis

15

- Para os demais exemplos de declaração de variáveis:
 - ▣ usar o simulador MARS
 - ▣ compilar o exemplo1.asm
 - ▣ verificar e entender e explicar a alocação de memória

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00350061	0x00000003	0x40600000	0x6c6c6548	0x6f57206f	0x0a646c72	0x00000000	0x0000002a
0x10010020	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x64636261	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000010	0x00000000	0x00000000	0x00000000	0x00000000

Adição, Subtração e Acesso à Memória

```
#Example 2 - Adição e subtração  
#exemplo f = (g + h) - (i + j);
```

```
.data  
  _f: .word 0  
  _g: .word 5  
  _h: .word 4  
  _i: .word 2  
  _j: .word 1  
.text  
.globl _main  
_main:  
  lw $s1, _g  
  lw $s2, _h  
  lw $s3, _i  
  lw $s4, _j  
  add $t0, $s1, $s2  
  add $t1, $s3, $s4  
  sub $s0, $t0, $t1  
  sw $s0, _f  
  li $v0, 10  
  syscall
```

Adição, Subtração e Acesso à Memória

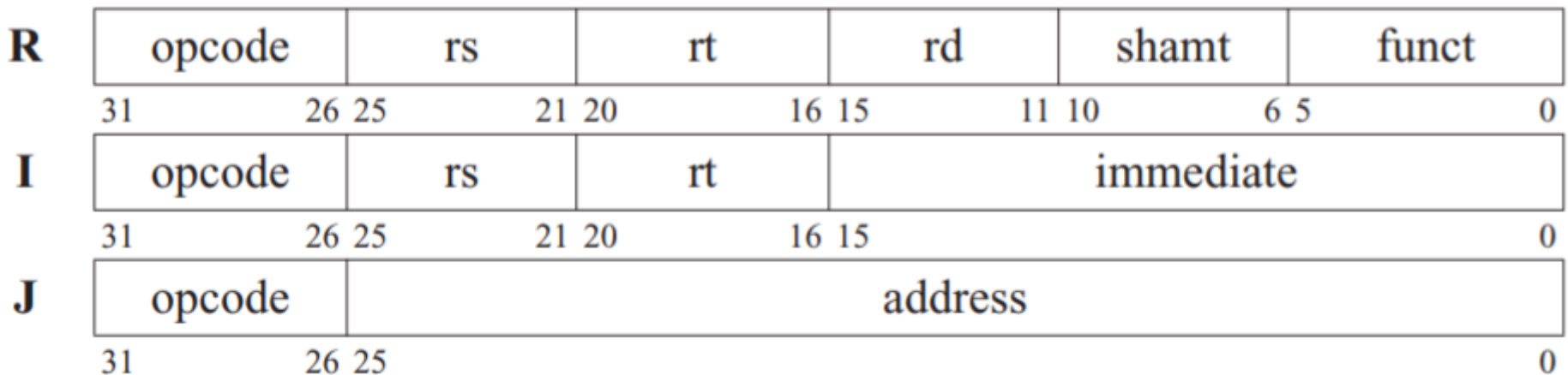
17

- Rodar exemplo2.asm no MARS:
 - ▣ Observar as mudanças que o assembler introduz
 - ▣ Observar os comandos `lui` e `lw`
 - ▣ Observar os cálculos e a evolução dos valores dos registradores utilizados
 - ▣ Observar o uso do comando `li`
 - ▣ E do registrador `$v0` e do comando `syscall`
 - ▣ Observar a coluna Code do simulador que mostra o código de máquina das instruções
 - `lui, lw, add, sub, sw` e `addiu`

Linguagem de Máquina

18

- Continuando com o exemplo2.asm vamos focar na linguagem de máquina.
- Os formatos básicos de instrução do MIPS são:



- Usaremos para isso a tabela de instruções do MIPS
- O MIPS Green Sheet do livro texto

Linguagem de Máquina

19

Formatos básicos de instrução

R	opcode	rs	rt	rd	shamt	funct
	31	26 25	21 20	16 15	11 10	6 5 0
I	opcode	rs	rt	immediate		
	31	26 25	21 20	16 15	0	
J	opcode	address				
	31	26 25	0			

Load Upper Imm. lui I $R[rt] = \{imm, 16'b0\}$

f_{hex}

Code: 0x3c011001

Binário: 0011 1100 0000 0001 0001 0000 0000 0001

Campos: 001111 00000 00001 0001 0000 0000 0001

opcode rs rt immediate

0x0f 0₁₀ 1₁₀ (\$at) 0x1001

{imm, 16'b0}: 0001 0000 0000 0001 0000 0000 0000 0000

$R[rt] = R[\$at] = 0x10010000$

Linguagem de Máquina

20

Formatos básicos de instrução

R	opcode	rs	rt	rd	shamt	funct
	31	26 25	21 20	16 15	11 10	6 5 0
I	opcode	rs	rt	immediate		
	31	26 25	21 20	16 15	0	
J	opcode	address				
	31	26 25	0			

Load Word lw I $R[rt] = M[R[rs] + \text{SignExtImm}]$ (2) 23_{hex}

Code: 0x8c310004

Binário: 1000 1100 0011 0001 0000 0000 0000 0100

Campos: 100011 00001 10001 0000 0000 0000 0100

opcode	rs	rt	immediate
0x23	1 ₁₀	17 ₁₀ (\$s1)	0x0004

R[rs]: 0001 0000 0000 0001 0000 0000 0000 0000 (0x10010000)

SignExtImm: 0000 0000 0000 0000 0000 0000 0000 0100 (0x00000004)

R[rt] = M[0x10010004] = 0x00000005

Linguagem de Máquina

21

Formatos básicos de instrução

R	opcode	rs	rt	rd	shamt	funct
	31	26 25	21 20	16 15	11 10	6 5 0
I	opcode	rs	rt	immediate		
	31	26 25	21 20	16 15	0	
J	opcode	address				
	31	26 25	0			

Add add R $R[rd] = R[rs] + R[rt]$ (1) 0 / 20_{hex}

Code: 0x02324020

Binário: 0000 0010 0011 0010 0100 0000 0010 0000

Campos: 000000 10001 10010 01000 00000 100000

opcode rs rt rd shamt funct

0x00 17₁₀ 18₁₀ 8₁₀ 0₁₀ 0x20

R[rs]: 0000 0000 0000 0000 0000 0000 0000 0101 (0x00000005)

R[rt]: 0000 0000 0000 0000 0000 0000 0000 0100 (0x00000004)

R[rd]: 0000 0000 0000 0000 0000 0000 0000 1001 (0x00000009)

Linguagem de Máquina

22

Formatos básicos de instrução

R	opcode	rs	rt	rd	shamt	funct
	31	26 25	21 20	16 15	11 10	6 5 0
I	opcode	rs	rt	immediate		
	31	26 25	21 20	16 15	0	
J	opcode	address				
	31	26 25	0			

Store Word sw **I** $M[R[rs] + \text{SignExtImm}] = R[rt]$ (2) $2b_{\text{hex}}$

Code: 0xac300000

Binário: 1010 1100 0011 0000 0000 0000 0000 0000

Campos: 101011 00001 10000 0000 0000 0000 0000

opcode	rs	rt	immediate
0x2b	1_{10}	$16_{10} (\$s0)$	0x0000

$R[rs]:$ 0001 0000 0000 0001 0000 0000 0000 0000 (0x10010000)

SignExtImm: 0000 0000 0000 0000 0000 0000 0000 0000 (0x00000000)

$M[0x10010000] = R[rt] = 0x00000006$

Operações lógicas

#Example 3 - Operações lógicas (sll, srl, and, andi, or, ori, nor)

```
.data
.text
.globl _main
_main:
    #Carrega os valores imediatos nos REGISTRADORES
    li $s0, 0x00000009
    li $s1, 0x00003c00
    li $s2, 0x00000dc0
    #Realiza o calculo
    sll $t0, $s0, 4      #0x00000090
    srl $t1, $t0, 2      #0x00000024
    and $t2, $s1, $s2    #0x00000c00
    or  $t3, $s1, $s2    #0x000003dc0
    andi $t4, $s2, 255   #(255 16bits)0x00ff => 0x000000c0
    ori  $t5, $s2, 15    #(15 16bits)0x000f  => 0x00000dcf
    nor  $t6, $s1,$zero  # not $s0 => 0xffffc3ff
    #Termina o programa
    li $v0, 10
    syscall
```

Operações lógicas

24

- Observar o resultado de cada operação do exemplo3.asm e tentar compreender como ele foi produzido usando com o comando em questão.
- Comandos presentes no código:
 - ▣ sll - Shift left
 - ▣ srl - Shift right
 - ▣ and - And bit a bit
 - ▣ andi - And bit a bit (op. imediato)
 - ▣ or - Or bit a bit
 - ▣ ori - Or bit a bit (op. imediato)
 - ▣ nor - Nor bit a bit (usado como not no exemplo)

Tomada de decisão

#Example 4 - Condicionais (Tomada de decisão)

#exemplo if (i == j) f = g + h; else f = g - h;

```
.data
    _f: .word 0
    _g: .word 5
    _h: .word 4
    _i: .word 2
    _j: .word 1
.text
.globl _main
_main:
    #Carrega os valores das variáveis nos registradores MEM -> REG
    lw $s1, _g
    lw $s2, _h
    lw $s3, _i
    lw $s4, _j
    #Realiza o calculo
    bne $s3,$s4,Else    #se i!=j => Else
    add $s0,$s1,$s2     #f=g+h (se i!=j pula, se i==j executa)
    j Exit              #salto incondicional para Exit
Else:
    sub $s0,$s1,$s2     #se i!=j, f=g-h
Exit:
    #Retorna o valor do resultado para a memória REG -> MEM
    sw $s0, _f
    #Termina o programa
    li $v0, 10
    syscall
```


Tomada de decisão

26

- ❑ Rodar o exemplo4.asm observando:
- ❑ O valor do `$pc` antes e depois da execução do `bne`
- ❑ Igualar o valor de `i` e `j` e rodar novamente o exemplo
- ❑ Continuar observando o valor de `$pc`

Tomada de decisão

27

- Observar o uso dos labels `Else` e `Exit`
- perceba como eles são substituídos no código por valores numéricos
- Tente explicar os valores usados no lugar dos labels
- E como o novo valor de `$pc` é produzido a partir deles
- Use a tabela de instruções do MIPS para mais informações

Linguagem de Máquina

28

R	opcode	rs	rt	rd	shamt	funct
	31 26 25	21 20	16 15	11 10	6 5	
I	opcode	rs	rt	immediate		
	31 26 25	21 20	16 15			
J	opcode	address				
	31 26 25					

Branch On Not Equal `bne`

I if($R[rs] \neq R[rt]$)
PC = PC + 4 + BranchAddr

(4) 5_{hex}

(4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }

14 bits como valor igual ao do bit 15 do immd.

Code: 0x16740002

Binário: 0001 0110 0111 0100 0000 0000 0000 0010

Campos: 000101 10011 10100 0000 0000 0000 0010

opcode rs rt immediate

0x05 19_{10} 20_{10} 0x0002 (2_{10})

R[pc]: 0000 0000 0100 0000 0000 0000 0010 0000 (0x00400020)

R[pc]+4: 0000 0000 0100 0000 0000 0000 0010 0100 (0x00400024)

BranchAddr: 0000 0000 0000 0000 0000 0000 0000 1000 (0x00000008)

R[pc]+4+BranchAddr: 0x0040002c

Linguagem de Máquina

29

R	opcode	rs	rt	rd	shamt	funct
	31 26 25	21 20	16 15	11 10	6 5	
I	opcode	rs	rt	immediate		
	31 26 25	21 20	16 15			
J	opcode	address				
	31 26 25					

Jump j J PC=JumpAddr (5) 2_{hex}

(5) JumpAddr = { PC+4[31:28], address, 2'b0 }

4 bits mais sig. Do PC+4

Code: 0x0810000c

Binário: 0000 1000 0001 0000 0000 0000 0000 1100

Campos: 000010 00 0001 0000 0000 0000 0000 1100

opcode

address

0x02

0x010000c

R[pc]: 0000 0000 0100 0000 0000 0000 0010 1000 (0x00400028)

R[pc]+4: 0000 0000 0100 0000 0000 0000 0010 0100 (0x0040002c)

JumpAddr: 0000 0000 0100 0000 0000 0000 0011 0000 (0x00400030)

R[pc] = 0x00400030

Laços de repetição

#Example 5 - Laços de repetição

#exemplo while(save[i]==k) i+=1;

```
.data
    _i: .word 0
    _k: .word 2
    _save: .word 2, 2, 3, 4, 5      # Array de words (inteiros)
.text
.globl _main
_main:
    #Carrega os valores das variáveis nos registradores MEM -> REG
    lw $s3, _i
    lw $s5, _k
    la $s6, _save
    #Realiza o calculo
Loop:
    sll $t1,$s3,2      # $t1=i*4 (offset)
    add $t1,$t1,$s6    # $t1=$s6+$t1 (endereço)
    lw $t0,0($t1)      # $t0=save[i]
    bne $t0,$s5,Exit   # Se save[i]!=k => Exit
    addi $s3,$s3,1      # $s3=$s3+1 (incremento)
    j Loop              # Salta de volta para Loop
Exit:                  # Termina o laço
    #Termina o programa
    li $v0, 10
    syscall
```

Laços de repetição

#Example 6 - Laços de repetição

#exemplo for(i=0;i<5;i++){ a[i]+=5; }

```
.data
    _i: .word 0
    _a: .word 1, 2, 3, 4, 5          # Array de words (inteiros)
.text
.globl _main
_main:
    #Carrega os valores das variáveis nos registradores MEM -> REG
    lw $s1, _i
    li $t0, 5
    la $s6, _a
    #Realiza o calculo
loop:
    beq $s1, $t0, end    # se i==10 termina
    sll $t2, $s1, 2      # $t2=i*4 (offset)
    add $t2, $t2, $s6    # $t2=$s6+$t2 (endereço)
    lw $t3, 0($t2)       # $t3=a[i]
    addi $t3, $t3, 5     # $t3+=5 (inc. de a[i])
    sw $t3, 0($t2)       #
    addi $s1, $s1, 1     # i+=1 (inc. de i)
    j loop               # Salta pra o teste
end:
    #Termina o programa
    li $v0, 10
    syscall
```